

Competitive Programming

Appendix A: Lazy Propagation

Competitive Programming Course

2026-07-10

Table of contents

1	Why a normal segment tree is not enough	3
2	Vocabulary	4
3	Worked example: range add and range sum	5
4	Tag composition	6
5	Recursive template	7
6	Proof habit	8
7	C++ and Python notes	9
8	Common mistakes	10
9	Exercises	11
10	Before you submit	12

1 Why a normal segment tree is not enough

A point-update segment tree changes one leaf and recomputes its ancestors. If an operation says “add x to every element in $[l, r]$ ”, walking to every leaf is $O(r-l+1)$. Repeating that for many full-range updates is quadratic.

Lazy propagation stores an update at the highest node whose entire segment is covered. It changes that node’s aggregate immediately and postpones touching its children until a later operation needs to distinguish them.

2 Vocabulary

For a node representing segment I :

- `tree[node]` is the aggregate for the values in I , including every update already applied to all of I .
- `lazy[node]` is a tag still owed to both children.
- **apply** updates one node's aggregate and composes its pending tag.
- **push** applies the pending tag to both children before descending.
- **pull** recomputes a parent aggregate after changing its children.

The central invariant is:

```
tree[node] equals the aggregate of its whole segment after all updates.  
lazy[node] describes exactly the updates not yet copied to its children.
```

3 Worked example: range add and range sum

Start with [2, 1, 3, 4] and add 5 to [1,4]. At the root, whose segment has length four, update its sum from 10 to 30 and record lazy add 5. There is no reason to visit either child yet.

Now query [1,2]. The root does not fit entirely in that query, so push its tag. Each child gets its aggregate increased by $5 * \text{child_length}$ and stores lazy add 5. The left child now answers $2+1+10 = 13$; the root still stores the correct total 30.

The multiplication by segment length is why a sum tree needs more than a plain difference-array idea.

4 Tag composition

Tags are operations, not values. If an older tag **old** is waiting and a newer full-cover update **new** arrives, a child later needs the effect of applying **old** and then **new**.

Update type	Node aggregate	Compose pending tag
add x , range sum	<code>sum += x * length</code>	<code>add += x</code>
assign x , range sum	<code>sum = x * length</code>	replace pending assignment
flip bit, count ones	<code>ones = length - ones</code>	<code>flip ^= 1</code>

Assignment and addition do **not** commute. If supporting both, the tag must record whether an assignment exists and the later addition after it. Pushing in the wrong order is a logical error even when samples pass.

5 Recursive template

```
update(node, left, right, ql, qr, tag):
    if [left,right] is inside [ql,qr]:
        apply(node, tag, right-left+1)
        return
    push(node, left, right)
    recurse into intersecting children
    pull(node)

query(node, left, right, ql, qr):
    if [left,right] is inside [ql,qr]: return tree[node]
    push(node, left, right)
    return merge(recurse into intersecting children)
```

Never push at a leaf. Never pull before both changed children have their correct aggregates.

6 Proof habit

Prove **apply** first: it must make the node aggregate correct for its full segment and compose the exact deferred child operation. Then prove **push**: both children become correct for their full segments, so a recursive update/query can rely on the invariant. Finally, **pull** is correct because the parent aggregate is the merge of two correct child aggregates. Induction on recursion therefore proves every operation correct.

7 C++ and Python notes

In C++, keep `tree` and `tags` in `vector<long long>` and pass segment endpoints to `apply`. In Python, recursion depth is safe for a balanced tree but use lists, not dictionaries, for predictable speed. In both languages use a single indexing convention: the local Section 37 exercise is 1-based.

8 Common mistakes

- Update a sum by `x` instead of `x * segment_length`.
- Push after recursing, so children answer stale values.
- Forget to clear a tag after pushing it.
- Treat assignment and addition as commutative tags.
- Query a partial range without pushing first.

9 Exercises

1. **Range add/range sum:** solve Section 37 C. State why an add tag composes by addition.
2. **Range assign/range sum:** implement a tag with an optional assignment. Test assigning the same range twice and assigning a child after a parent.
3. **Binary flip/count:** use xor tag composition. Test two flips of the same range and overlapping flips.

10 Before you submit

```
[ ] I can state what each tag means in chronological order.  
[ ] apply updates both the aggregate and the deferred tag.  
[ ] push happens before a partial descent.  
[ ] pull happens after every changed child returns.  
[ ] A brute-force array agrees on random ranges.
```